Graph Based PDCoEA for Solving Min-Max Problems

Supervised by: Dr. Per Kristian Lehre

Siddharth Mukundan

University of Birmingham

August 13, 2026

Outline

Motivation

- Co-Evolutionary Algorithms were developed to address the intractability of finding Nash Equilibriums using Classical Algorithms for Certain Problems.
- The Pairwise Dominant Co-Evolutionary Algorithm (PDCoEA) was a successful implementation with various empirical results and a theoretically proven runtime for the bilinear problem.
- Can we do better?

Motivation

- We believe that the Graph-Based Pairwise Dominant Co-Evolutionary Algorithm could perform better than the Vanilla Version.
- This Algorithm maintains a diversity of initial populations on separate nodes and interactions between the populations are modeled by edges connecting these nodes.
- The diversity could lead to more exploration across the fitness landscape and find the solution quicker.

Problem Statement: Definitions

Graph-based Population Structure

Directed graph $G = (V, E)$ with $V = \{(P_t^1, Q_t^1), \ldots, (P_t^n, Q_t^n)\}$, where (P_t^i, Q_t^i) are predator/prey populations of size λ at node $i \in [n]$, time t. Interactions are restricted to neighbors in E.

Target Set & Convergence

Let S be the set of desirable final states. Node j converges at time t if $(P_t^j \times Q_t^j) \cap S \neq \emptyset$.

Empirical Comparison

We instantiate G with various **topologies** (ring, star, line, complete) and compare against the **vanilla** PDCoEA ($n = 1$) on the DefendIt game.

Algorithm Overview

Algorithm 1 Graph-Based PDCoEA

Initialize digraph $G = (V, E)$
for all $v_i \in V$ **do**
    Sample $P_0^i(k), Q_0^i(k) \sim \text{Unif}(\{0,1\}^n)$ for all $k \in [\lambda]$
end for
for $t \in \mathbb{N}_0$ **do**
    Sample $e_{ij} \sim \text{Unif}(E)$
    for $k \in [\lambda]$ **do**
        Sample $(x_1, y_1), (x_2, y_2) \sim \text{Unif}(P_t^i \times Q_t^j)$
        $(x, y) \leftarrow (x_1, y_1)$ if $(x_1, y_1) \succeq_g (x_2, y_2)$, else (x_2, y_2)
        Set $P_{t+1}^i(k)$, $Q_{t+1}^j(k)$ by flipping bits of x, y with probability χ/n
    end for
end for

Experimental Setup: DefendIt

- DefendIt is a cryptographic multi-resource management game where an attacker and defender compete for ownership of multiple resources within a fixed budget.

Formal Definition

An instance of DefendIt is a tuple $(k, \ell, v, c, B^D, B^A)$, where $k \in \mathbb{N}$ is the number of resources, $\ell \in \mathbb{N}$ the number of time-steps, $v = (v^{(1)}, \ldots, v^{(k)})$ the resource values, $c = (c^{(1)}, \ldots, c^{(k)})$ the resource costs, B^D the defender's budget, and B^A the attacker's budget.

Table 1: Ownership outcome for a single resource

Defender	Attacker	Outcome
0	0	Previous owner (unchanged)
0	1	Attacker owns
1	0	Defender owns
1	1	Previous owner (unchanged)

Experimental Setup: DefendIt Cost & Payoff

Cost Function

$$C(x) := \sum_{j=1}^{k} c^{(j)} \sum_{i=1}^{\ell} x_i^{(j)}$$

Payoff (Defender)

$$g_1(x,y) := \begin{cases} \sum_{j=1}^{k} v^{(j)} \sum_{i=1}^{\ell} z_i^{(j)}(x,y) & \text{if } C(x) \leq B^D \\ -\sum_{j=1}^{k} \sum_{i=1}^{\ell} x_i^{(j)} & \text{otherwise} \end{cases}$$

Payoff (Attacker)

$$g_2(x,y) := \begin{cases} \sum_{j=1}^{k} v^{(j)} \left(\ell - \sum_{i=1}^{\ell} z_i^{(j)}(x,y)\right) & \text{if } C(y) \leq B^A \\ -\sum_{j=1}^{k} \sum_{i=1}^{\ell} y_i^{(j)} & \text{otherwise} \end{cases}$$

Experimental Setup: Champion Selection

- Since DefendIt is intractable, there is no standard baseline to compare our algorithms with. The payoff values are also subjective based on the attacker and defender populations in each algorithms.
- To address this, we use the Champion Selection method, where we compare the payoffs of the defender (attacker) population of algorithm $\mathcal{A}$ against the attacker (defender) population of algorithm $\mathcal{B}$.

Champion Selection: Formal Definitions

Period Champions

Defender champions in period i:

$$u_i^{\mathcal{A}} := \arg\max_{x \in P^{\mathcal{A}}_{T_i^{\mathcal{A}}}} \min_{y \in Q^{\mathcal{B}}_{T_i^{\mathcal{B}}}} g_1(x,y), \qquad u_i^{\mathcal{B}} := \arg\max_{x \in P^{\mathcal{B}}_{T_i^{\mathcal{B}}}} \min_{y \in Q^{\mathcal{A}}_{T_i^{\mathcal{A}}}} g_1(x,y)$$

Attacker champions defined analogously via g_2, giving $v_i^{\mathcal{A}}, v_i^{\mathcal{B}}$.

Combined Champion Pools

$$U_i := \bigcup_{t=0}^{i} \{u_t^{\mathcal{A}}, u_t^{\mathcal{B}}\}, \qquad V_i := \bigcup_{t=0}^{i} \{v_t^{\mathcal{A}}, v_t^{\mathcal{B}}\}$$

Relative Performance

$$D_i^{\mathcal{A}} := \max_{x \in P^{\mathcal{A}}_{T_i^{\mathcal{A}}}} \min_{y \in V_i} g_1(x,y), \qquad A_i^{\mathcal{A}} := \max_{y \in Q^{\mathcal{A}}_{T_i^{\mathcal{A}}}} \min_{x \in U_i} g_2(x,y)$$

($D_i^{\mathcal{B}}, A_i^{\mathcal{B}}$ defined symmetrically.)

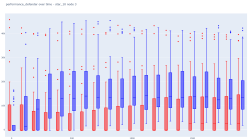
Experimental Setup: Parameters

Parameter	Value
Population size λ	50
Mutation rate χ	0.3
Resources k	10
Time-steps ℓ	40
Generations t	2000
Champion period τ	100
Repeats per topology	30
Defender budget B^D	28,000
Attacker budget B^A	28,000

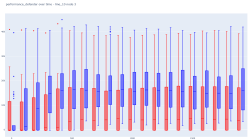
Table 2: Experiment parameters

Results: Defender Performance vs. Vanilla

Ring vs. Vanilla



Star vs. Vanilla



Line vs. Vanilla

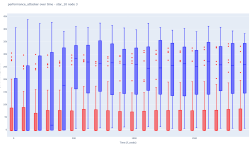
Complete vs. Vanilla

Observations

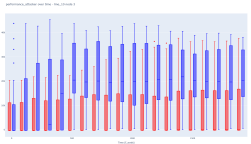
Across the four topologies, the Vanilla PDCoEA Algorithm outperforms the other topologies, suggesting that cross-node interactions are detrimental to the convergence of the algorithm. The Vanilla version also exhibits a substantial amount of variance but the graph versions produce more outliers overall, with the Ring and Star topologies producing consistent outliers throughout the runs.

Results: Attacker Performance vs. Vanilla

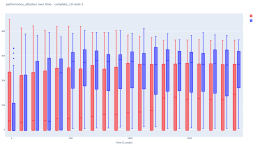
Ring vs. Vanilla



Star vs. Vanilla



Line vs. Vanilla



Complete vs. Vanilla

Observations

Attacker performances mirror the defender performances, except for the number of outliers which appears to be reduced as compared to the defender plots.

Conclusion

- Graph-PDCoEA performs significantly worse than the Vanilla PDCoEA when it comes to DefendIt, reflected by the attacker and defender pay-off values. A reason for this could be that allowing a node to interact with multiple nodes could cause positive and negative drifts that cancel each other out, leaving the populations fixed.
- One possible way to fix this would be to change the sampling strategy of the edges which prioritizes exploitation during the later stages by sampling an edge more frequently, thus converging to an optimum.

Future Work

- Proving theoretically that Graph-PDCoEA attains zero drift in certain configurations.
- Varying the edge-sampling strategy and performing empirical studies

Thank You! Questions?